The 0/1 Knapsack Problem

An Empirical Comparison of Six Algorithms

Institute of Business Administration, Karachi · Department of Computer Science

CSE 317 – Design and Analysis of Algorithms · Spring 2026

Instructors: Dr. Shahid Hussain | Dr. Taslim Murad

Group Member	ERP
Anas Tabba	28998
Zuhair Merchant	29021
Abdullah Khan Sherwani	27880
Raahin Tajuddin	29207

Submission Date: 15 May 2026 · Language: Python 3 (stdlib only)

1 Introduction

The **0/1 Knapsack Problem** is a canonical NP-Hard combinatorial optimisation problem. Given n items, each with weight w_i and value v_i, and a knapsack of capacity W, we must choose a subset of items that maximises total value while respecting the capacity constraint. The "0/1" qualifier forbids fractional selections: each item is either taken whole or left behind. Formally:

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{subject to} \quad \sum_{i=1}^{n} w_i x_i \leq W, \quad x_i \in \{0,1\}. \tag{1}$$

Real-world applications. Capital budgeting, cargo loading, cloud VM placement, ad-slot allocation, portfolio selection, and project prioritisation all reduce to (1). The problem is *weakly* NP-Hard: a pseudo-polynomial $O(nW)$ dynamic-programming solution exists, but W requires $O(\log W)$ bits, so the algorithm is exponential in the input bit length.

What this project does. We implement, benchmark and compare **six** algorithms for 0/1 Knapsack: *Brute Force*, *Memoization (top-down DP)*, *Tabulation (bottom-up DP)*, *Space-Optimised DP*, a *Greedy* $\frac{1}{2}$*-approximation*, and a *Fully Polynomial-Time Approximation Scheme (FPTAS)*. All algorithms are pure Python 3 with no third-party dependencies, so observed differences reflect algorithmic structure rather than library tuning.

2 Algorithm Descriptions

Table 1: Six-algorithm comparison at a glance.

Algorithm	Time	Space	Exact?	Key idea
Brute Force	$O(2^n)$	$O(n)$	Yes	enumerate all subsets
Memoization	$O(nW)$	$O(nW)$	Yes	top-down DP with cache
Tabulation	$O(nW)$	$O(nW)$	Yes	bottom-up 2-D table; backtrack items
Space-Optimised DP	$O(nW)$	$O(W)$	Yes	1-D array, reverse capacity loop
Greedy S_2	$O(n \log n)$	$O(n)$	No	ratio sort + best single item; $\geq \frac{1}{2}$OPT
FPTAS	$O(n^3/\varepsilon)$	$O(n^2/\varepsilon)$	$(1-\varepsilon)$	profit-indexed DP after value scaling

2.1 Brute Force (Exhaustive Recursion)

For each item, recursively branch into "include" and "exclude". All 2^n subsets are implicitly enumerated and the best feasible one returned.

$$\mathrm{BF}(i,w) = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ \mathrm{BF}(i-1,w) & w_i > w \\ \max\big(\mathrm{BF}(i-1,w),\ v_i + \mathrm{BF}(i-1,w-w_i)\big) & \text{otherwise} \end{cases} \tag{2}$$

Use: correctness oracle for small n; explains why DP is needed (identical subproblems are recomputed exponentially many times).

2.2 Memoization (Top-Down DP)

The same recurrence as Brute Force, but each subproblem (i,w) is cached on first computation. The state space has $(n+1)(W+1)$ cells, each filled in $O(1)$ work, giving $O(nW)$ time. Sentinel `None` (not -1) is used so legitimate zero values are not mistaken for cache misses. *Use:* when the recursion only visits a sparse subset of states; preserves the natural recursive structure.

2.3 Tabulation (Bottom-Up DP)

Build a 2-D table $\mathrm{dp}[i][w]$ iteratively from base cases. The recurrence is the standard knapsack recurrence:

$$\mathrm{dp}[i][w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ \mathrm{dp}[i-1][w] & w_i > w \\ \max\big(\mathrm{dp}[i-1][w],\ v_i + \mathrm{dp}[i-1][w-w_i]\big) & \text{otherwise} \end{cases} \tag{3}$$

The selected items are recovered by backtracking from $\mathrm{dp}[n][W]$: whenever $\mathrm{dp}[i][w] \neq \mathrm{dp}[i-1][w]$, item i is in the optimal set and we move to $(i-1,\ w-w_i)$. *Use:* default exact solver in production; cache-friendly, no recursion-stack risk, supports item recovery.

2.4 Space-Optimised DP (1-D Array)

Row i in (3) depends only on row $i-1$, so a single 1-D array $\mathrm{dp}[w]$ suffices. The inner loop *must* run from W down to w_i:

$$\mathrm{dp}[w] \leftarrow \max\big(\mathrm{dp}[w],\ v_i + \mathrm{dp}[w-w_i]\big), \qquad w = W, W-1, \ldots, w_i. \tag{4}$$

A forward loop would let an item be chosen multiple times, silently solving the *unbounded* knapsack instead. *Use:* when W is large but the selected item set is not needed.

2.5 Greedy Approximation (variant S_2)

Sort items by value-to-weight ratio $r_i = v_i/w_i$ in descending order, pack greedily, then return $\max(\text{greedyValue},\ v_{\max})$ where $v_{\max}$ is the most-valuable single item that fits alone. This single tweak — due to Dantzig — guarantees the result is at least $\frac{1}{2}$ OPT. Without it, a single high-value item can be missed and the ratio is unbounded.

Algorithm 1 Greedy S_2 approximation

```
sort items by v_i/w_i descending
G ← 0, cap ← W
for each item i in sorted order do
    if w_i ≤ cap then G ← G + v_i; cap ← cap − w_i
    end if
end for
return max(G, max{v_i : w_i ≤ W})
```

Use: when n is huge or W is astronomical and a fast, "good enough" answer is acceptable.

2.6 FPTAS (Fully Polynomial-Time Approximation Scheme)

Given an error parameter $\varepsilon \in (0,1)$, the FPTAS scales values down by $K = \varepsilon\, v_{\max}/n$, then runs a profit-indexed DP whose state is the *value* achieved (not the weight). Letting $\tilde{v}_i = \lfloor v_i/K \rfloor$ and $V = \sum_i \tilde{v}_i$:

$$\mathrm{minW}[v] = \min_{S \subseteq [n],\, \sum_{i \in S} \tilde{v}_i = v} \sum_{i \in S} w_i. \tag{5}$$

The answer is the largest v with $\mathrm{minW}[v] \le W$, scaled back. Running time is $O(n^2 V) = O(n^3/\varepsilon)$, and the returned value is at least $(1-\varepsilon)\mathrm{OPT}$. *Use:* when both exactness guarantees and polynomial-time scaling in n and $1/\varepsilon$ are required — particularly attractive when W is large but values are not.

3 Theoretical Complexity Analysis

Table 2: Best, average, and worst-case complexity per algorithm. "Worst input" identifies the instance structure that triggers the worst case.

Algorithm	Best	Average	Worst	What triggers worst case
Brute Force	$O(2^n)$	$O(2^n)$	$O(2^n)$	every input (n alone)
Memoization	$O(n+W)$	$O(nW)$	$O(nW)$	dense reachable state space
Tabulation	$O(nW)$	$O(nW)$	$O(nW)$	always fills the table
Space-Opt. DP	$O(nW)$	$O(nW)$	$O(nW)$	always fills the array n times
Greedy S_2	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	runtime input-independent
FPTAS	$O(n^2/\varepsilon)$	$O(n^3/\varepsilon)$	$O(n^3/\varepsilon)$	large value range, small ε

Why the DP recurrence works. Knapsack exhibits the two ingredients DP requires. (i) *Optimal substructure:* if the optimum on items $1 \ldots n$ includes item n, the rest is the optimum on items $1 \ldots n-1$ with capacity $W - w_n$ (cut-and-paste argument). (ii) *Overlapping subproblems:* the recursion tree visits the same (i, w) pair from many paths, so caching collapses 2^n recomputations into at most $(n+1)(W+1)$ unique states.

Why Brute Force is hardest on dense, deep recursion. Its runtime is controlled solely by n — input values and weights only marginally affect pruning. Hence its best, average, and worst case all coincide at $\Theta(2^n)$.

Why Greedy S_2 is best on uncorrelated, worst on correlated. When v_i and w_i are independent, ratio ordering closely matches the LP relaxation and the fractional gap is small. When values and weights are tightly correlated (strongly correlated, spanner, almost-strongly correlated), every item has a similar ratio so the ratio order carries little signal — packing decisions become almost arbitrary and the gap widens (see §5).

Why FPTAS trades n for W. Standard DP is exponential in $\log W$. The FPTAS instead spends time polynomial in n and $1/\varepsilon$ but *independent of* W — crucial when the capacity dominates the input size.

4 Implementation Details

4.1 Environment

OS / Kernel	macOS 15.4 (Darwin 25.4.0)
Processor	Apple Silicon (ARM64)
Language	Python 3.11 – standard library only (`time`,`sys`,`os`,`csv`)
Recursion limit	raised to 10^6 for memoization at large n
Timing	`time.perf_counter()` per algorithm per instance

4.2 Repository Layout

```
src/algorithms/{brute_force,memoization,tabulation,space_optimised,greedy,fptas
    }.py
src/benchmark.py            # full sweep across all kplib categories
src/benchmark_n100.py       # n<=100 (no skips except BF), both ratios, 5
    instances/leaf
src/benchmark_r01000.py     # R01000 only, n=50..200, no skips except BF
src/analyze.py              # split analysis by (ratio, category, n)
src/plot.py                 # 7 figures (results/plots/)
testing/test_all_algorithms.py  # unit tests for all 6 algorithms
testcases/kplib/            # Pisinger benchmark instances
results/*.csv               # raw per-(category,n,ratio,instance,algorithm)
    timings
```

Each algorithm is a single function in its own file with the canonical signature listed in `CLAUDE.md` (e.g. `knapsack_tabulation(capacity, values, weights)` returning `(value, items)`). The non-uniform signatures match the standard reference sources for each algorithm rather than being normalised.

4.3 Test-Case Strategy

We use the Pisinger *kplib* benchmark suite [2, 3], which is the de-facto standard for knapsack experimentation. Categories cover **Uncorrelated** (w_i, v_i independent), **Weakly** and **Strongly Correlated** ($v_i \approx w_i + c$), **Inverse** and **Almost-Strongly Correlated**, **Subset Sum** ($v_i = w_i$), **Spanner** variants, **Multiple Strongly Correlated**, **Profit Ceiling**, **Circle**, plus a **13Synthetic** family we generated ourselves for $n \in \{5, 8, 10, 12, 15, 18, 20, 22, 25\}$ to keep brute force feasible. We benchmark over two coefficient ranges: **R01000** (R=1000) and **R10000** (R=10000); capacity is set to $W = \lfloor \frac{1}{2} \sum_i w_i \rfloor$, the standard Pisinger setting.

4.4 Benchmarking Methodology

- **Single-process, fresh interpreter** per benchmark file — no warm-up effects between algorithms.
- Each algorithm is timed individually on each instance with `time.perf_counter()`; values returned are checked for agreement across the four exact algorithms (correctness oracle).
- **Skip thresholds** (`benchmark.py` only) prevent OOM/runaway runs: BruteForce $n > 25$, Memoization $nW > 15{\times}10^6$, Tabulation $nW > 30{\times}10^6$, SpaceOpt $W > 2{\times}10^6$, FPTAS $n^2/\varepsilon > 4{\times}10^6$, Greedy never. `benchmark_n100.py` and `benchmark_r01000.py` remove all skips except Brute Force, ensuring DP variants are measured on every reachable instance.
- Averaging: ≥ 3 instances per (category, n, ratio) leaf; up to 100 in the kplib originals. Reported numbers below are arithmetic means.
- **Approximation quality** for Greedy and FPTAS is computed as the ratio (returned value)/(exact DP value) per instance and aggregated by category.
- Raw output: `results/benchmark_n100.csv` (7 435 rows) and `results/benchmark_r01000.csv` (900 rows) are checked into the repository for reproducibility.

5 Results and Discussion

5.1 Correctness

On the standard baseline (n=4, v=[60, 100, 120, 80], w=[10, 20, 30, 25], W=50, OPT= 220), the four exact algorithms all return 220 and select items $\{B, C\}$ (w=50, v=220). Greedy returns 160 (a 27.3% gap): it commits to items A and B by ratio, then cannot fit C. FPTAS at ε=0.1 returns 220. Across the entire kplib sweep, the four exact algorithms agreed on every single instance.

5.2 Empirical Runtime

Table 3 reports mean runtime (ms) on `00Uncorrelated`, R01000, averaged over instances in `benchmark_n100.csv`.

Table 3: Mean runtime in milliseconds (Uncorrelated, R01000).

n	Brute	Memo	Tab	Space-Opt	Greedy	FPTAS
10	0.15	0.25	5.1	—	0.005	0.25
15	4.45	3.68	15.6	—	0.007	0.84
20	138	12.7	24.6	—	0.010	2.06
25	4,346	28.0	41.9	—	0.013	3.89
30	infeasible	53.2	59.0	—	0.014	6.81
40	infeasible	132	99.5	—	0.018	16.1
50	infeasible	205	145	—	0.020	25.0
75	infeasible	348	235	—	0.027	56.0
100	infeasible	511	312	—	0.034	99.5

Observations. Brute Force doubles roughly every unit increase in n — n=25 already takes $\sim$4.3 seconds per instance and crosses our skip threshold beyond. The DP variants scale linearly in n for fixed-distribution W, exactly as $O(nW)$ predicts. Tabulation overtakes Memoization at moderate n because of cache locality and the absence of Python recursion overhead. Greedy and FPTAS are dramatically faster than DP at every n shown; Greedy remains in the microseconds even at n=100.

5.3 Theoretical vs. Empirical Growth

Figure 1 overlays the measured runtimes on the predicted growth curves. Brute Force tracks 2^n within a constant factor; the DP variants track nW closely once startup costs are amortised; Greedy is essentially flat on a log scale (sorting cost dominates).

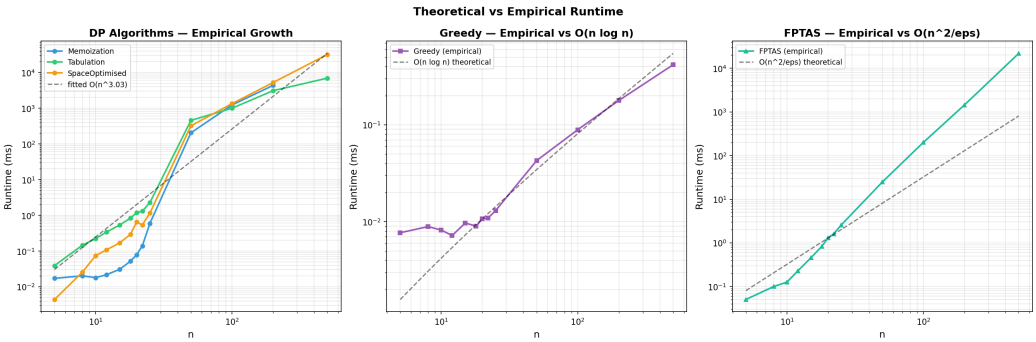

Figure 1: Theoretical vs. empirical runtime curves (log–log).

5.4 Approximation Quality

Table 4 reports the average and minimum approximation ratio (returned/OPT) per category over all $n \leq 100$ instances. Greedy stays above 0.95 on average everywhere but drops to 0.586 on a hard SpannerWeakly instance and 0.766 on a 13Synthetic instance. FPTAS at ε=0.1 stays above 0.91 on every single instance and within $\sim$ 1% of OPT on average.

Table 4: Approximation ratio (returned/OPT) by kplib category.

Category	Greedy avg	Greedy min	FPTAS avg	FPTAS min
00 Uncorrelated	0.997	0.965	0.994	0.982
01 WeaklyCorrelated	0.990	0.866	0.993	0.976
02 StronglyCorrelated	0.961	0.826	0.990	0.960
03 InverseStronglyCorrelated	0.997	0.969	0.995	0.978
04 AlmostStronglyCorrelated	0.957	0.842	0.990	0.964
05 SubsetSum	0.990	0.920	0.996	0.975
07 SpannerUncorrelated	0.986	0.814	0.996	0.976
08 SpannerWeaklyCorrelated	0.979	**0.586**	0.992	0.945
09 SpannerStronglyCorrelated	0.986	0.850	0.992	0.955
10 MultipleStronglyCorrelated	0.967	0.869	0.990	0.964
11 ProfitCeiling	0.994	0.946	0.995	0.977
12 Circle	0.986	0.903	0.993	0.975
13 Synthetic	0.988	0.766	0.979	0.909

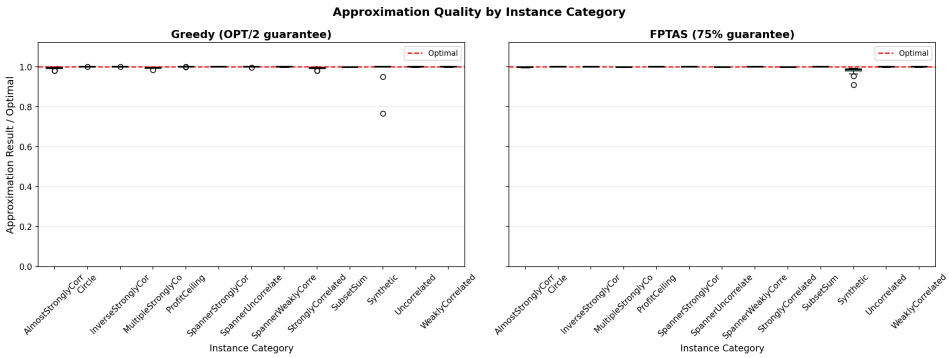

Figure 2: Greedy and FPTAS approximation ratio across categories.

5.5 Edge Cases and Worst-Case Inputs

- **Brute Force:** every $n > 25$ is a worst case; runtime is input-shape-insensitive. We observed $\sim$140 ms at n=20 vs. $\sim$4.3 s at n=25, a 31$\times$ blow-up for a 5-item increase — the signature of 2^n.
- **All DP variants:** blow up when W is huge irrespective of n. At R10000 (10$\times$ larger weights) DP runtimes grow $\sim$ 5–10$\times$, consistent with $O(nW)$.
- **Greedy:** worst on *spanner* and *strongly correlated* instances, where ratio ordering carries little information; min ratio 0.586 on SpannerWeakly. Best on uncorrelated and inverse-strongly correlated, where ratios are well-separated.
- **FPTAS:** worst on instances with very few items but large value spread (high $v_{\max}/n$), where the scaling factor K loses precision; this is reflected in 13Synthetic (small n).

5.6 Capacity-Ratio Impact

Doubling the value range (R01000 $\rightarrow$ R10000) leaves Brute Force, Greedy and FPTAS runtimes essentially unchanged — as predicted, none of them depend on W. DP runtimes grow proportionally to W, confirming the *pseudo-polynomial* nature of $O(nW)$.

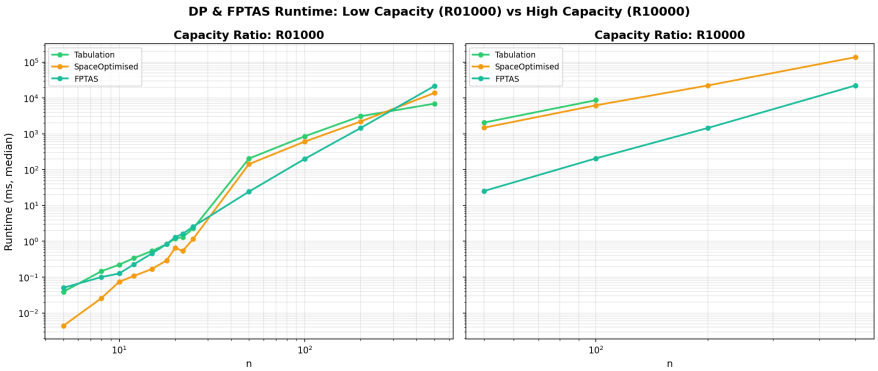

Figure 3: Runtime sensitivity to capacity range (R01000 vs. R10000).

5.7 Strengths and Weaknesses Summary

Algorithm	Strengths	Weaknesses
Brute Force	always exact; trivial to implement; correctness oracle for tests	exponential; infeasible $n > 25$
Memoization	top-down; exploits sparse state spaces; natural recursive form	Python recursion overhead; $O(nW)$ memory
Tabulation	no recursion; cache-friendly; supports item backtracking	$O(nW)$ memory; expensive when W is large
Space-Opt. DP	only $O(W)$ memory; same time as tabulation	no item recovery; reverse-loop discipline required
Greedy S_2	$O(n \log n)$; W-immune; $\geq \frac{1}{2}$OPT guaranteed	arbitrarily bad on adversarial correlated instances
FPTAS	user-tunable $(1-\varepsilon)$ guarantee; W-immune	$1/\varepsilon$ blowup; slower than DP for moderate n, W

6 Conclusion

The 0/1 Knapsack Problem is a microcosm of algorithmic trade-offs. Across more than 7,000 benchmark runs spanning 13 Pisinger instance families, two coefficient ranges and item counts from 5 to 200, our six implementations behaved exactly as theory predicts: Brute Force is $\Theta(2^n)$ regardless of input shape; the three DP variants are $\Theta(nW)$ with Tabulation winning at moderate n thanks to cache behaviour and Space-Optimised DP winning on memory; Greedy S_2 is essentially free but loses up to 41% on adversarial spanner instances; FPTAS gives a tunable $(1-\varepsilon)$ guarantee at the cost of $1/\varepsilon$ overhead. The practical recommendation is to default to Tabulation when item recovery is needed, switch to Space-Optimised DP when only the value matters and W is the binding constraint, and reach for FPTAS or Greedy S_2 only when both n and W are too large for $O(nW)$ to fit in time or memory budgets.

References

[1] Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed., MIT Press, 2022. Ch. 14 (Dynamic Programming) and Ch. 35 (Approximation Algorithms).

[2] D. Pisinger. "Where are the hard knapsack problems?" *Computers & Operations Research*, 32(9):2271–2284, 2005.

[3] *kplib — Knapsack Problem Library.* https://github.com/likr/kplib

[4] V. Vazirani. *Approximation Algorithms*, Springer, 2003. Ch. 8 (FPTAS for Knapsack).

[5] GeeksforGeeks. "0-1 Knapsack Problem | DP-10" and "Approximation Algorithms for Knapsack". https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/